

# **Kapittel 14**

## **Hierarkiske dataformater**

# Læringsmål

- Kjenne til bruksområder for JSON og XML.
- Kunne bruke JSON og XML for å representere data.
- Kunne definere lovlig oppbygging av JSON- og XML-data med henholdsvis JSON Schema og XML Schema.
- Kunne bruke spørrespråk mot JSON og XML.
- Forstå hvordan JSON og XML kan lagres i databaser og brukes som utvekslingsformat mellom systemer.

# JSON = JavaScript Object Notation

- **Tekstlig** dataformat for å representere komplekse data
- Godt egnet for å beskrive **semi-strukturerte** data, altså delvis strukturerte data
- Godt egnet som **utvekslingsformat**, blant annet mye brukt i web-løsninger
- Mange **API**-er (Application Programming Interface), eller «programmeringsgrensesnitt», leverer data på JSON-format
- Mange **åpne datasett**, f.eks. fra yr.no eller data.norge.no, er på JSON-format
- JSON-data er enkelt å behandle i mange **programmeringsspråk**, blant annet JavaScript.

# JSON-verdier

- **Grunnleggende** verdier: tall, tekst, true/false, null
- Kan brukes i **nøkkel/verdi-par**

```
"tittel": "Salmer fra kjøkkenet"
```

- **Objekter** er lister av nøkkel/verdi-par:

```
{  
  "tittel": "Salmer fra kjøkkenet",  
  "år": 2003  
}
```

- **Arrays** er lister av verdier

```
[1, 3, 5, 7, 9]
```

```
["Norsk", "Svensk"]
```

# Sammensatt JSON-objekt

```
{  
  "tittel": "Salmer fra kjøkkenet",  
  "utgivelsesår": 2003,  
  "regissør": { "fornavn": "Bent", "etternavn": "Hamer" },  
  "sjanger": ["Komedie", "Drama"],  
  "handling": "Filmen handler om en svensk forsker som blir sendt til  
    Norge for å observere kjøkkenvanene til enslige menn. Han utvikler  
    et uventet vennskap med en av de mennene han skal observere.",  
  "skuespillere": [  
    { "fornavn": "Joachim", "etternavn": "Calmeyer", "rolle": "Isak" },  
    { "fornavn": "Tomas", "etternavn": "Norström", "rolle": "Folke" },  
    { "fornavn": "Bjørn", "etternavn": "Floberg", "rolle": "Grant" }  
  ],  
  "produksjonsselskap": "Bulldog Film",  
  "språk": ["Norsk", "Svensk"],  
  "varighet": 95  
}
```

**Objekter og arrays og enkle verdier**  
kan altså **kombineres** for å beskrive  
**komplekse datastrukturer**.

# Eksempel på bruk av syntaksreglene

Bruk av syntaksreglene for å danne et JSON-objekt som beskriver en person med to telefonnumre:

value  $\rightarrow$  object  $\rightarrow$  { pair , pair }

$\rightarrow$  { string : value, string : value }

$\rightarrow$  { string : value, string : [ value, value ] }

$\rightarrow$  { string : string, string : [ number, number ] }

$\rightarrow$  { "fornavn" : "Rania" , "tel" : [ 77665544, 22334455 ] }

# Databasetabeller på JSON-format

```
[
  {
    "KNr": 5002,
    "Fornavn": "Paal",
    "Etternavn": "Aass",
    "Adresse": "Lindemans gate 79",
    "PostNr": "1711"
  },
  {
    "KNr": 5007,
    "Fornavn": "Joakim",
    "Etternavn": "Laursen",
    "Adresse": "Thomas Heftyes gate 39",
    "PostNr": "0015"
  }
]
```

«**Rett fram**» oversettelse fra databasetabell til JSON:

- Hvert **tabellrad** blir til et **JSON-objekt**, der **kolonnenavn** blir til **nøkler**.
- Alle «**rad-objektene**» settes sammen i en **JSON-array**.

Men JSON har **flere muligheter...**

# Embedding

```
{
  "KNr":5321,
  "Navn":{
    "Fornavn":"Sivert",
    "Etternavn":"Ose"
  },
  "Adresse":"Harald Halvorsens vei 17",
  "PostNr":"0208",
  "Poststed":"Oslo",
  "Ordre":[
    {
      "OrdreNr":20571,
      "OrdreDato":"2024-09-24",
      "Ordrelinjer":[
        {
          "VNr":"49921",
          "Betegnelse":"Lezlo leire, økonomipakke",
          "PrisPrEnhet":514.00,
          "Antall":1
        },
        {
          "VNr":"64509",
          "Betegnelse":"Kongelilje, 10 stk.",
          "PrisPrEnhet":204.00,
          "Antall":5
        }
      ]
    }
  ]
}
```

Denne JSON-verdien viser 1 **kunde** med listen av **ordrer** for denne kunden «bakt inn» (**embedded**). I vårt eksempel er det også kun 1 ordre.

I (hver) ordre er listen med **ordrelinjer** embedded, i vårt eksempel 2 stykker.

Merk også **sammensatt attributt** Navn og **kontrollert redundans** med PostNr og Poststed.



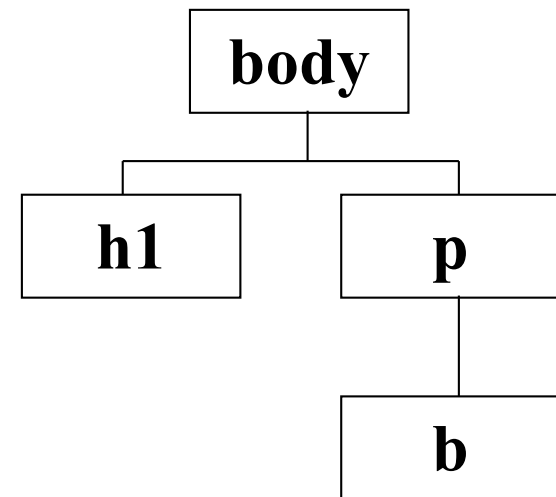
# Oppbygging av XML-dokumenter

- XML har «HTML-syntaks»

```
<?xml version="1.0" encoding="UTF-8"?>  
<melding dato="07.12.2024"  
        rom="5-116">  
    <avsender fnavn="Kari" enavn="Lie"/>  
    <beskjed>Møte om 5 min!</beskjed>  
</melding>
```

# HTML vs XML - I

```
<body>
  <h1>HTML</h1>
  <p>
    <b>HTML</b> er et språk
    for å beskrive nettsider.
  </p>
</body>
```



- HTML er bygd opp fra elementer i en tre-struktur
- Lovlige HTML-elementer er forhåndsdefinert
  - Nettlesere vet hva **h1**, **p** og **b** betyr
- HTML beskriver både **struktur** og **presentasjon**

# HTML vs XML - II

- XML er **utvidbart** (extensible)
  - Ingen forhåndsdefinerte elementer
  - Må tolkes for å bli presentert

```
<sjakktrekk>
```

```
  <brikke>
```

```
    <farge>hvit</farge>
```

```
    <type>tårn</type>
```

```
  </brikke>
```

```
  <fra>D1</fra>
```

```
  <til>D8</til>
```

```
</sjakktrekk>
```

.xml

→ .htm

→ .htm

→ .txt

- Kan vises som teksten **Hvitt tårn fra D1 til D8**, eller som en animasjon.

# XML - eXtensible Markup Language

- Syntaksregler for **velformet** XML
  - Hvert XML-dokument har ett og bare ett **rot-element**
  - Alle elementer har både start-tag og slutt-tag
  - Hvert element kan ha et antall attributter (ikke to attributter med samme navn)
  - Kan ha nøstede elementer som i HTML
- XML er et **metaspråk** = språk for å definere nye språk
  - Finnes for mange fagområder og bransjer, f.eks:

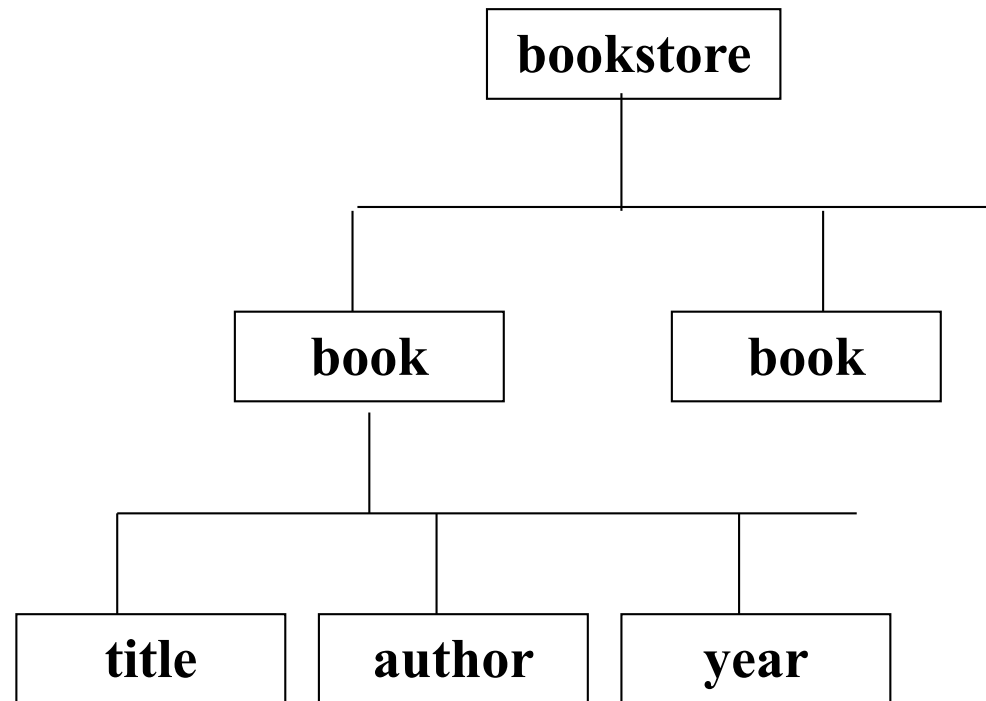
|                                 |                                                                |
|---------------------------------|----------------------------------------------------------------|
| Geografiske data ( <b>GML</b> ) | <a href="http://www.opengis.net/gml/">www.opengis.net/gml/</a> |
| e-handel ( <b>ebXML</b> )       | <a href="http://www.ebxml.org">www.ebxml.org</a>               |
| Kjemiske formler ( <b>CML</b> ) | <a href="http://www.xml-cml.org">www.xml-cml.org</a>           |

# XPath

- For å lage regler som «skriver om» XML-dokumenter er det nyttig å kunne snakke om deler av et XML-dokument
  - Eksempel: Innholdet av elementet <title> som befinner seg under første forekomst av elementet <book>
- **XPath** er en notasjon for å referere til deler av et XML-dokument
  - XPath er basert på **stiuttrykk**
  - XPath inneholder et bibliotek av standardfunksjoner
  - XPath er en W3C Standard

# Trestrukturen

- Foreldre, barn, søsken
- Forfedre, avkom



# XML og databaser

- Anvendelser
  - XML for dynamiske data til web
  - XML som overføringsformat
- Lagre XML i databaser
  - CLOB eller XMLType
  - Last inn data
- Spørringer mot XML-data
  - XPath
  - XQuery
- Produsere XML fra tabelldata

# Strukturerte data i XML variant 1

- XML-representasjon av radsamlinger
  - Rot-element = tabellnavn
  - Elementnavn = navn på databasekolonne
  - Tekst i elementet (mellom taggene) = verdi

```
<?xml version="1.0" encoding="UTF-8"?>
```

```
<Studenter>
```

```
  <Student>
```

```
    <Studentnr>1</Studentnr>
```

```
    <Navn>Ole Hansen</Navn>
```

```
  ...
```

```
</Student>
```

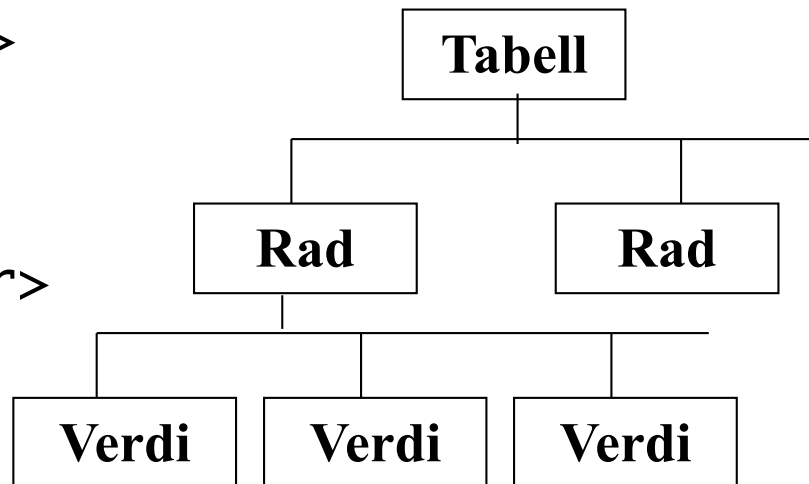
```
<Student>
```

```
  <Studentnr>2</Studentnr>
```

```
...
```

```
</Student>
```

```
</Studenter>
```





# Strukturerte data i XML variant 2

- Alternativ: Data i attributter

```
<Student Studnr="012345", Fornavn="Ole",  
        Etternavn="Hansen", Gate="Bøgata",  
        Husnr="23", Postnr="3800",  
        Poststed="Bø">  
</Student>
```

- Elementnavn = tabellnavn
- Attributtnavn = navn på databasekolonne
- Attributtverdi = verdi

# XML Dokumenter i PostgreSQL

```
CREATE TABLE testXML  
(  
  id serial,  
  data xml  
);
```

1



```
INSERT INTO testXML VALUES (2, XMLPARSE (DOCUMENT '
```

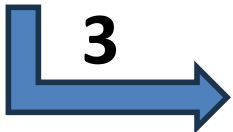
```
<breakfast_menu>  
<food>  
  <name>Strawberry Belgian Waffles</name>  
  <price>$7.95</price>  
  <description>  
    Light Belgian waffles covered with strawberries and whipped cream  
  </description>  
  <calories>900</calories>  
</food>  
<food>  
  <name>Berry-Berry Belgian Waffles</name>  
  <price>$8.95</price>  
  <description>  
    Light Belgian waffles covered with an assortment of fresh berries  
    and whipped cream  
  </description>  
  <calories>900</calories>  
</food>
```

2



```
SELECT CAST  
(xpath('/breakfast_menu/food[name  
e="French Toast"]', DATA) AS  
TEXT[]) AS NAME FROM testXML WHERE  
id=2;
```

3



|   | name<br>text[]                                                                                                                                                                                      |
|---|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | { "<food><br><name>French Toast</name><br><price>\$4.50</price><br><description><br>Thick slices made from our homemade sourdough bread<br></description><br><calories>600</calories><br></food>" } |

```
<food>  
  <name>French Toast</name>  
  <price>$4.50</price>  
  <description>  
    Thick slices made from our homemade sourdough bread  
  </description>  
  <calories>600</calories>  
</food>
```

```
));
```

# Semistrukturerte data

- Karakteristikk
  - Ikke tabellstruktur, mer en **dokumentstruktur**
  - Skjemafrie (følger ikke gitt typestruktur)
- Behov
  - Ønsker å lagre slike data i «databaser»
  - Ønsker å kunne søke (ad hoc) med et spørrespråk
- Hvordan lagre semistrukturerte data?
  - Som XML-dokumenter i filer
  - Normalisert tabellstruktur i relasjonsdatabase
  - Hybridløsning: XML-dokumenter som verdier i relasjonsdatabaser
  - Objektreasjonelle databaser med XML-støtte
  - XML-databaser
  - NoSQL-databaser

# XML som utvekslingsformat

- Hvorfor bruke XML som utvekslingsformat?
  - Fleksibelt: Utvidbar syntaks
  - Selvbeskrivende (elementnavn)
  - «Åpent»: Lesbart for maskiner og mennesker